

Captured Flags and Lessons Learned: The Digital Hunt Continues

On June 25, 2026, I had the opportunity to compete in the global MetaCTF Competition.

It was an intense, 2-hour online event that challenged our hands-on technical skills under strict time constraints.

I successfully pwned the challenges detailed below.

This experience exposed me to critical technical layers and practical problem-solving methodologies that I had not encountered before.

In cybersecurity, every challenge is a classroom, and my learning journey continues!

FIRST CHALLENGE ON METACTF2026

MetaCTF / SkillBit Writeup 2026

- **Challenge Name:** Exposure
- **Category:** Forensics / Misc
- **Points:** 100

Challenge Description

A coworker dropped a photo (20251023_141508.jpg) into the team chat, posted straight off their phone. The picture is nothing special, but they weren't thinking about what else rode along with it.

Problem

Previous Submissions

A coworker dropped a photo (20251023_141508.jpg) into the team chat, posted straight off their phone. The picture is nothing special, but they weren't thinking about what else rode along with it.

Download the photo [here](#).

Please rate this problem: ☆☆☆☆☆

Step-by-Step Solution

Step 1: Downloading the File

First, I downloaded the challenge artifact, which was an image file named 20251023_141508.jpg.

Step 2: File Analysis

I used the file command to verify the file type and ensure it was a valid image. After confirmation, I used exiftool to inspect the metadata (EXIF data) embedded within the image:

Bash

```
file 20251023_141508.jpg
exiftool 20251023_141508.jpg
Use code with caution.
```

Step 3: Extracting the Flag

While analyzing the exiftool output, I closely inspected the metadata fields. I observed a hidden string inside the **User Comment** property, which contained the flag:

text

User Comment : SkillBit{str1p_y0ur_3x1f_b3f0r3_p0st1ng}

Use code with caution.

Flag: SkillBit{str1p_y0ur_3x1f_b3f0r3_p0st1ng}

Lessons Learned & Cybersecurity Impact

This challenge demonstrates the risks of **Data Leakage (Information Disclosure)** through metadata. When you take a photo with a smartphone, the device automatically attaches EXIF data. As shown in the terminal output, this metadata leaked:

1. **Device Specifications:** Phone model (Samsung Galaxy S23) and software version.
2. **Geographical Location:** Precise GPS coordinates (40 deg 0' 53.94" N, 105 deg 16' 13.97" W), which can be used for OSINT attacks to track a user's physical location.
3. **Sensitive Comments:** Hidden internal data (or flags, in this case).

Mitigation:

Always strip or sanitize EXIF metadata before uploading photos to public spaces or untrusted team chats. You can use tools like exiftool -All= image.jpg to clean the file.

Terminal Output Reference

bash

```
└─(masterⓂmaster)-[~/Downloads]
└─$ exiftool 20251023_141508.jpg
ExifTool Version Number      : 13.50
File Name                    : 20251023_141508.jpg
...
User Comment                  : SkillBit{str1p_y0ur_3x1f_b3f0r3_p0st1ng}
...
GPS Position                  : 40 deg 0' 53.94" N, 105 deg 16' 13.97" W
Use code with caution.
```

SECOND CHALLENGE IN METACTF 2026

- **Challenge Name:** Flash sale
- **Category:** Web exploitation
- **Points:** 200

Challenge Discription:

FlashCart just dropped their Founders Edition Hoodie. It's members-only, it's \$200, and every one ships with an exclusive promo code we want.

The catch: a fresh account starts with a \$0.00 balance, and the only gift card floating around (FLASH50) is worth \$50 and is limited to one redemption per account. Get yourself a hoodie.

URL Link: <https://1cea607fe559ea16.chals.sbhost.io>

Problem

Previous Submissions

FlashCart just dropped their Founders Edition Hoodie. It's members-only, it's \$200, and every one

The catch: a fresh account starts with a \$0.00 balance, and the only gift card floating around is worth \$50 and is limited to one redemption per account. Get yourself a hoodie.

Container spawned. Running for 13953 seconds.

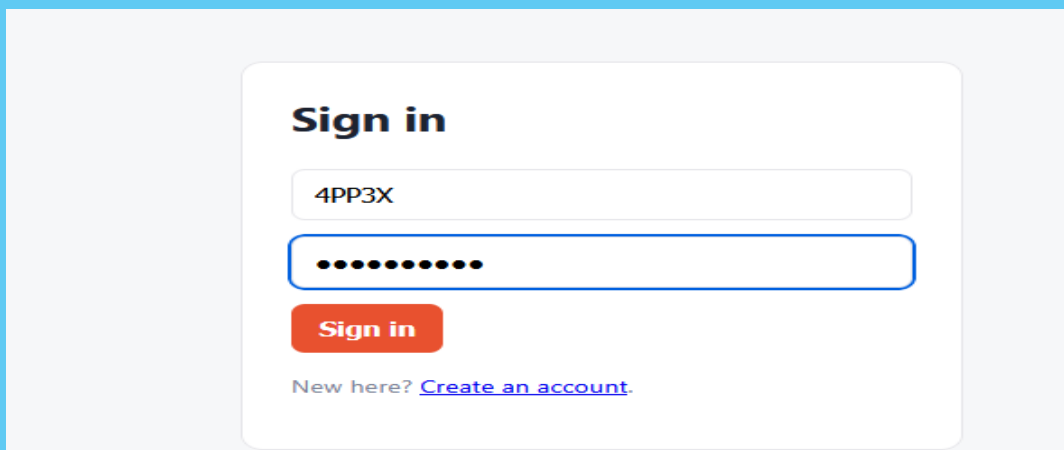
Delete

Service	Protocol	Endpoint
Flash Sale	HTTPS	https://1cea607fe559ea16.chals.sbhost.io

How did I solve this challenge to got Flag:

Step-by-Step Solution

Step 1: Account Creation & Reconnaissance



A sign-in form titled "Sign in" with a white background and rounded corners. It features a text input field containing "4PP3X", a password input field with ten dots, a red "Sign in" button, and a link "New here? [Create an account.](#)"

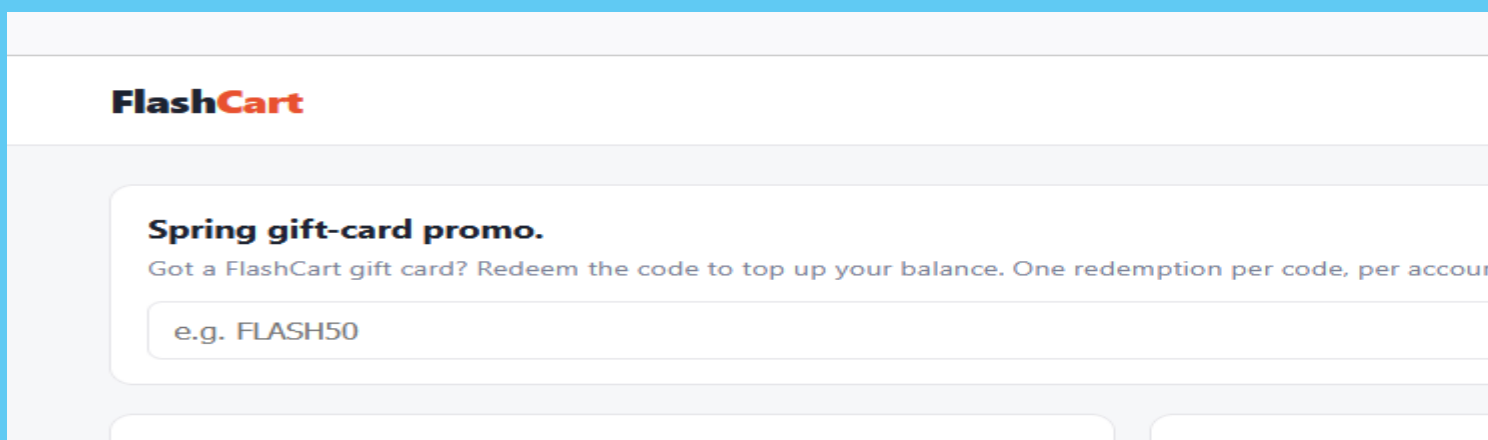
Sign in

4PP3X

.....

Sign in

New here? [Create an account.](#)



A promotional form for FlashCart. It has the "FlashCart" logo at the top. Below it is a section titled "Spring gift-card promo." with a description: "Got a FlashCart gift card? Redeem the code to top up your balance. One redemption per code, per account." There is a text input field with the placeholder "e.g. FLASH50".

FlashCart

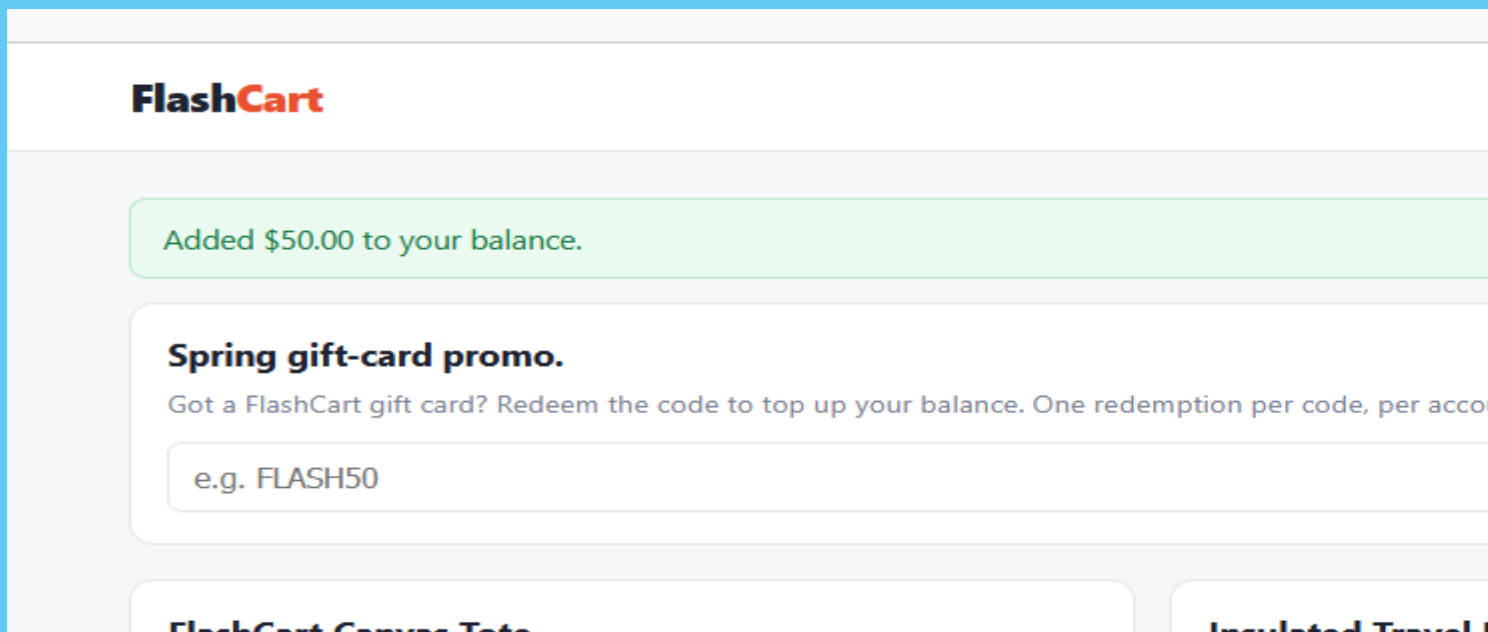
Spring gift-card promo.

Got a FlashCart gift card? Redeem the code to top up your balance. One redemption per code, per account.

e.g. FLASH50

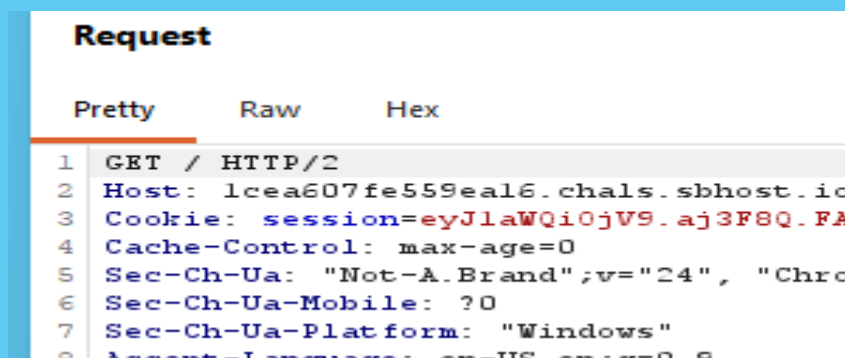
I opened the provided URL in the browser. The application required authentication, so I registered a new account and logged in using my created credentials. Upon logging in, I observed the following:

- Initial Account Balance: **\$0.00**
- Target Item Price (Founders Edition Hoodie): **\$200.00**
- Available Promo Code: FLASH50 (Adds \$50.00, but limited to **one redemption per account**)



Step 2: Intercepting the Request

To bypass the single-redemption restriction, I turned on **Burp Suite** to intercept the HTTP request generated when applying the FLASH50 coupon code. I captured the POST request responsible for redeeming the gift card and sent it to the **Burp Suite Repeater**.

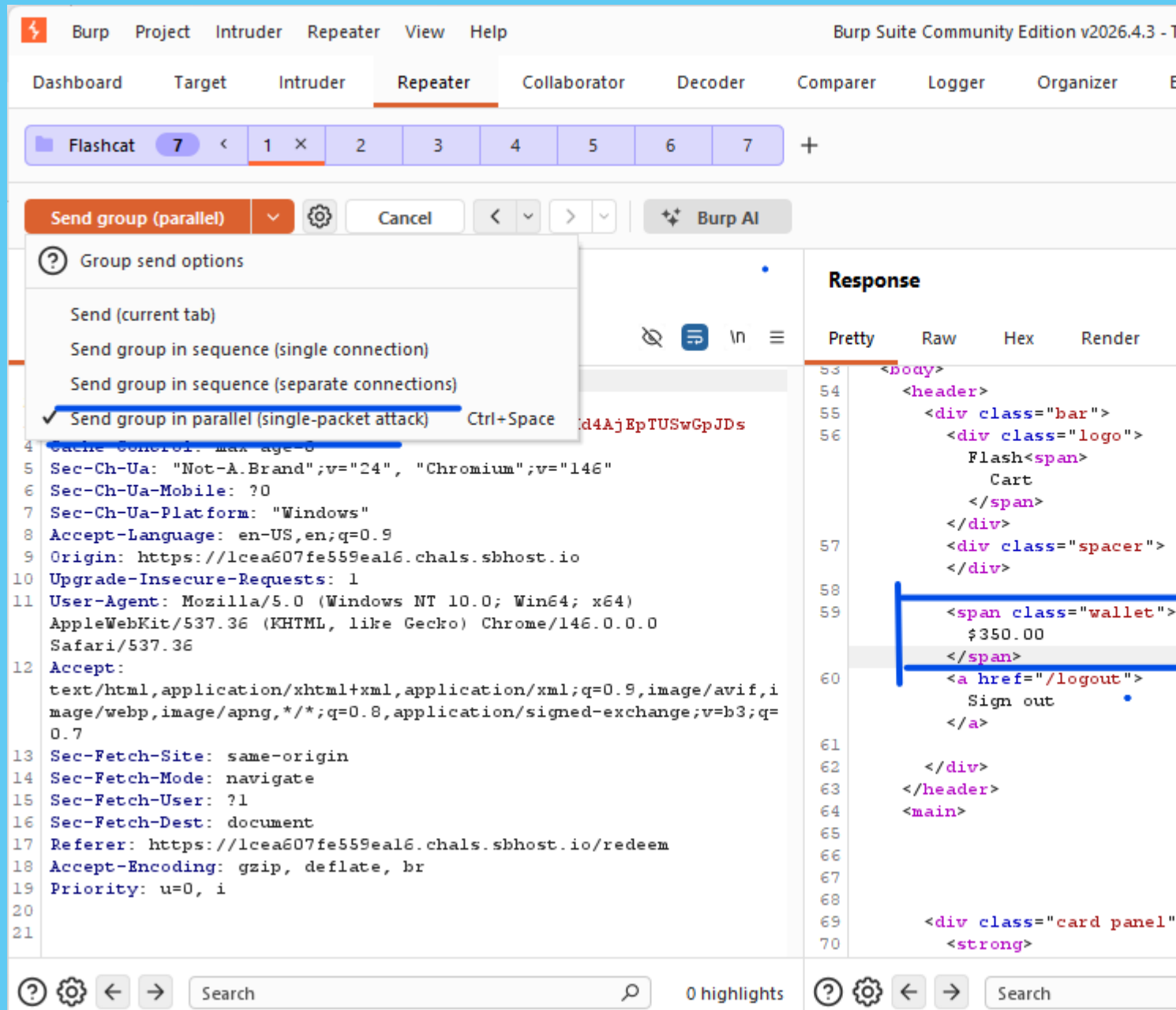


Step 3: Executing the Race Condition Attack (TOCTOU)

Since the backend checks if the coupon is used *before* updating the database, I exploited a **Time-of-Check to Time-of-Use (TOCTOU)** race condition flaw.

1. In Burp Repeater, I created a **Request Group** containing multiple identical redemption requests.
2. I configured the attack to use the **Parallel-Single Attack** engine (sending all requests simultaneously in a single connection to bypass network latency).

3. Upon executing the parallel attack, the server processed multiple requests concurrently before it could update the database state to mark the coupon as "used."

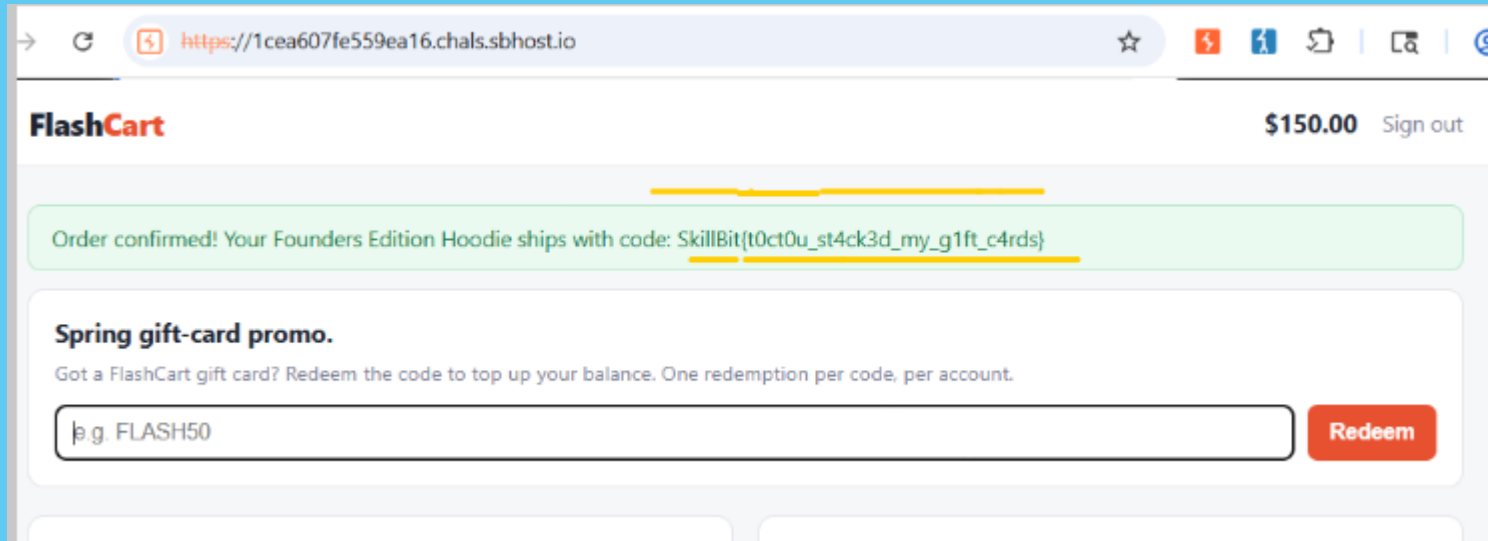


Step 4: Claiming the Flag

I turned off Burp Suite interception and refreshed my browser page (F5). The race condition was successful: the single \$50 coupon was processed multiple times, inflating my account balance to **\$350.00**.

With a balance exceeding the \$200 threshold, I successfully purchased the Founders Edition Hoodie. The application then revealed the flag in the response.

Flag: SkillBit{t0ct0u_st4ck3d_my_g1ft_c4rds}



Lessons Learned & Cybersecurity Impact

1. What is TOCTOU / Race Condition?

This challenge highlights a critical business logic flaw known as **Race Condition (CWE-367: Time-of-Check to Time-of-Use)**. It occurs when a web application processes concurrent requests in a multi-threaded environment without proper synchronization.

The server checks if the user has used the coupon (Time-of-Check), sees "No", and then applies the money. If 7 requests arrive at the exact same millisecond, the server says "No" to all 7 before any of them can change the status to "Yes" (Time-of-Use).

2. Real-World Impact

In e-commerce platforms, TOCTOU vulnerabilities can lead to financial loss, unauthorized inventory depletion, or double-spending bugs in financial applications and digital wallets.

4. Mitigation (How to Fix It)

To prevent this vulnerability, developers must implement **Atomic Transactions** and **Database Locking**:

- **Row-Level Locking:** Use SQL queries like `SELECT ... FOR UPDATE` to lock the user's coupon status row until the transaction completes.
- **Concurrency Control:** Implement synchronized blocks or strict rate-limiting per account for sensitive transaction endpoints.

THIRD CHALLENGE IN METACTF2026

MetaCTF / SkillBit Writeup 2026

- **Challenge Name:** Key Evidence
- **Category:** Digital Forensics / Hardware
- **Points:** 150

Challenge Description

Our DLP team pulled a USB capture off the kiosk that staff use to unlock the records terminal. The only device on the bus was the keyboard. We need to know exactly what got typed at it.

Problem

Previous Submissions

Our DLP team pulled a USB capture off the kiosk that staff use to unlock the records terminal. We need to know exactly what got typed at it.

Download the artifact [here](#).

Please rate this problem: ☆ ☆ ☆ ☆ ☆

How did i got this flag:

Step-by-Step Solution

Step 1: Initially Analysis

First, I inspected the downloaded file (keyboard.pcap) using the file command to determine the type of traffic captured:

bash

file keyboard.pcap

Use code with caution.

Output:

text

keyboard.pcap: pcap capture file, microsecond ts (little-endian) - version 2.4 (Memory-mapped Linux USB, capture length 65535)

```
master@master: ~/Downloads

(master@master)-[~/Downloads]
$ tshark -r keyboard.pcap -Y "usb.transfer_type == 0x01 && usb.capdata" -T fields -e usb.c

(master@master)-[~/Downloads]
$ ls
20251023_141508.jpg  ctf_env  image.jpg  Metactf
'archive(1).tar'     eu-central-1-4PP3X-regular.ovpn  keyboard.pcap  picoctf_lab
archive.tar          flag.enc  ld-linux-x86-64.so.2  'Screenshot F
cogwork              hex_codes.txt  libc.so.6        solve.py

(master@master)-[~/Downloads]
$ cat hex_codes.txt
000000000000000000
000000000000000000
000000000000000000
000000000000000000
020016000000000000
000000000000000000
00000e000000000000
000000000000000000
00000c000000000000
000000000000000000
00000f000000000000
```

Use code with caution.

The output confirmed that this was not ordinary network traffic (like HTTP or DNS), but a **USB packet capture** documenting communication between a keyboard and the system.

Step 2: Extracting the Raw USB HID Payloads

Because the keystrokes are transmitted as raw scancodes (**USB Human Interface Device protocol**) rather than plain text, standard utilities like strings or grep could not find the flag format directly.

To isolate the keyboard input data, I **used Tshark** to filter for USB interrupt data (usb.transfer_type == 0x01) and extract the specific payload field (usb.capdata):

```
bash
```

```
tshark -r keyboard.pcap -Y "usb.transfer_type == 0x01 && usb.capdata" -T fields -e usb.capdata > hex_codes.txt
```

Use code with caution.

This produced a list of 8-byte structures inside **hex_codes.txt** representing the raw state changes of the keyboard.

Step 3: Decoding Scancodes and Handling Backspaces

According to the USB HID keyboard specification:

- **Byte 0 (Modifier):** If this byte contains 0x02, the **Left Shift** key was held down.
- **Byte 2 (Key Code):** Represents the specific key pressed (e.g., 0x16 is 'S', 0x20 is '3').
- **Backspace Key (0x2a):** Tells the system to delete the previously typed character.

```
(master@master)-[~/Downloads]
└─$ cat hex_codes.txt
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0200160000000000
0000000000000000
00000e0000000000
0000000000000000
```

I used a custom **Python script (solve.py)** to parse hex_codes.txt, track when Shift was applied, and handle the backspace logic dynamically to reconstruct the exact string typed by the user:

```
bash
```

```
python3 solve.py
```

Use code with caution.

```

(master@master)-[~/Downloads]
$ nano solve.py

(master@master)-[~/Downloads]
$ python3 solve.py
Booom 4PP3X flag SkillBit{HID_r3p0rt5_l34k_3v3ry_k3y}

Please rate this problem: ☆☆☆☆☆

(master@master)-[~/Downloads]
$ _

```

Output:

text

Booom 4PP3X flag SkillBit{HID_r3p0rt5_l34k_3v3ry_k3y}

Use code with caution.

Flag: SkillBit{HID_r3p0rt5_l34k_3v3ry_k3y}

Lessons Learned & Cybersecurity Impact

1. The Risk of USB Passive Sniffing

USB architecture implicitly trusts peripheral devices connected to the bus. If an attacker plants a physical hardware sniffer or compromises the local operating system to log USB bus data, they can capture sensitive credentials (like the terminal unlock keys in this scenario) without triggering traditional network-based DLP alerts.

2. Forensic Reconstructive Logic

When analyzing HID captures, automation tools must account for dynamic typist behavior. Simply mapping codes directly to letters will yield incorrect flags if the user made typos and hit **Backspace (0x2a)**. Implementing a stack structure (like `list.pop()` in Python) ensures that deleted characters are correctly omitted from the final payload.

FOURTH METACTF WRITEUP CHALLENGE 2026

How did they Got the flag

MetaCTF / SkillBit Writeup 2026

- **Challenge Name:** Cogwork
- **Category:** Reverse Engineering

- **Points:** 250

Problem

Previous Submissions

The vault keypad doesn't run normal code. It hands your access code to a little clockwork engine inside the firmware and only opens if the gears line up. Recover the code it's waiting for.

Download the binary [here](#).

Please rate this problem: ☆☆☆☆

Challenge Description

The vault keypad doesn't run normal code. It hands your access code to a little clockwork engine inside the firmware and only opens if the gears line up. Recover the code it's waiting for.

Step-by-Step Solution

Step 1: Initial Static Analysis

The binary cogwork is a statically linked, stripped 64-bit ELF executable. Running the application directly prompts the user for an access code:

text

```
cogwork> hunter2
```

```
Access denied.
```

```
Use code with caution.
```

```
(ctf_env)~(master@master)-[~/Downloads]
$ file cogwork
strings cogwork | grep -i -E "flag|secret|key|wrong|correct"

cogwork: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, BuildID
a7e9d8, for GNU/Linux 3.2.0, stripped
WARNING: Unsupported flag value(s) of 0x%x in DT_FLAGS_1.
(mode_flags & PRINTF_FORTIFY) != 0
version == NULL || !(flags & DL_LOOKUP_RETURN_NEWEST)
Protocol wrong type for socket
correction >= 0
wrong ELF class: ELFCLASS32
Wrong medium type
Required key not available
Key has expired
Key has been revoked
Key was rejected by service
ENOKEY
EKEYEXPIRED
EKEYREVOKED
EKEYREJECTED
```

Opening the file in a decompiler (like Ghidra or IDA Pro) reveals that the main function is relatively short. Instead of standard string comparisons (strcmp), the binary utilizes a loop containing a massive switch statement operating on a single byte pulled from a constant array. This indicates that the binary implements a custom **Bytecode Virtual Machine (VM)** interpreter.

Step 2: Reversing the Virtual Machine Opcodes

By analyzing the branching pathways within the switch block, the custom instruction set and operations of the stack-based 8-bit VM were mapped out:

- 0x13 (LDIN): Loads user input at index i.
- 0x14 (LDK): Loads a constant from an 8-byte key table (KTAB).
- 0x15 (LDT): Loads a byte from the target validation table (TGT).
- 0x16 (XOR) / 0x17 (ADD) / 0x19 (ROL) / 0x1b (NEQ): Core mathematical and bitwise operations.
- 0x37 (RND): Generates data via /dev/urandom to create a noise prologue loop from address 0x0000 to 0x001f. This routine performs dummy math on scratch registers to break instruction-counting side-channel attacks before discarding the outputs.

Step 3: Isolating the Validation Logic

The meaningful cryptographic loop resides at address 0x0020. The VM processes each input byte using the following sequence:

1. Fetches input character at the current index: `input[i]`
2. Applies a Bitwise XOR with a key from an 8-byte array using an interleaved masking key: `input[i] ^ KTAB[i & 7]`
3. Adds the current loop index offset to index position variant: `+ i`
4. Performs an 8-bit circular bitwise left-rotation by 3 bits: `rol8(..., 3)`
5. Compares the mutated state against the target memory table: `== TGT[i]`

The mathematical abstraction of the forward transform loop is expressed as:

$$\backslash(\text{TGT}[i]=\text{rol8}((\text{input}[i]\oplus \text{KTAB}[i\mathinner{\{\backslash;\text{left}(\backslash\text{mod}\backslash,8\backslash\text{right})\}})+i,3)\backslash)$$

Crucially, any mismatch flags an accumulator register (`acc |= mismatch`) instead of exiting early. This provides **constant-time execution**, preventing single-byte brute-forcing via latency analysis.

Step 4: Extracting Constants and Inverting the Loop

To reverse this, the exact values of KTAB and TGT arrays were extracted directly from the `.rodata` section of the binary file using Ghidra's memory viewer:

- **KTAB** (8 bytes) = [246, 165, 118, 211, 192, 23, 218, 62]
- **TGT** (43 bytes) = [221, 14, 32, 173, 60, 66, 21, 98, 236, 244, 216, 125, 231, 147, 253, 131, 164, 15, 81, 157, 157, 225, 86, 195, 101, 175, 224, 223, 253, 34, 230, 97, 78, 103, 67, 86, 30, 252, 160, 4, 127, 127, 169]

By implementing the inverse operation — executing an 8-bit right-rotation (`ror8`), subtracting the loop index $\backslash(i \bmod \{256\}\backslash)$, and applying a secondary XOR inversion against the static key context — the access code flag was extracted using a custom Python script:

Python

```
#!/usr/bin/env python3
```

```
# Recovered from the binary's .rodata section
```

```
KTAB = [246, 165, 118, 211, 192, 23, 218, 62]
```

```
TGT = [221, 14, 32, 173, 60, 66, 21, 98, 236, 244, 216, 125, 231, 147, 253, 131, 164, 15, 81, 157, 157, 225, 86, 195, 101, 175, 224, 223, 253, 34, 230, 97, 78, 103, 67, 86, 30, 252, 160, 4, 127, 127, 169]
```

```
def ror8(x, r):
```

```
    x &= 0xff
```

```
    return ((x >> r) | (x << (8 - r))) & 0xff
```

```
flag = bytearray()
```

```
for i, t in enumerate(TGT):  
    x = ror8(t, 3)  
    x = (x - i) & 0xff  
    x ^= KTAB[i & 7]  
    flag.append(x)
```

```
print("Boom 4PP3X flag:", flag.decode())
```

Use code with caution.

Flag: MetaCTF{c0gw0rk_runs_0n_byt3c0d3_n0t_m4g1c}

Lessons Learned & Cybersecurity Impact

1. Obfuscation via Custom Bytecode Architectures

Compiling validation checks into a custom Virtual Machine architecture shifts the target layout away from standardized instruction sets (like x86_64 or ARM) towards proprietary instruction formats. This effectively breaks standard signature scanners and out-of-the-box decompilation logic, forcing reverse engineers to map the control structures manually.

2. Side-Channel Mitigation Techniques

The inclusion of non-short-circuiting logic (ORA instead of early breaks) combined with an entropy-driven prologue loop (/dev/urandom) presents an excellent architectural pattern for preventing side-channel attacks. By standardizing the instruction and execution count across all inputs, the application safely mitigates automated single-character differential timing analyses.

LEARNING IS A JOURNEY: THANK YOU ALL! 🧡🧡🧡🧡

As a cybersecurity student, I believe that our growth depends on how we face complex digital challenges every single day.

Our journey forward is driven by four core pillars:

- **Curiosity** – The hunger to understand how things work underneath.
- **Consistency** – Showing up and grinding through the code day by day.
- **Passion** – The inner fire that loves solving puzzles and breaking barriers.
- **Persistence** – Refusing to quit when the compilation fails or a flag is hidden.

These pillars are what make us move **one step ahead every day** in our technical expertise. Keep hunting, keep learning!

Prepared by:

- **Author:** Frank Karani
- **Course:** Cybersecurity Student
- **Level:** First-Year Student (IAA)